

Name : M Kavya

Reg. No : 24011101061

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

Actual split used: Train+val = 3680 images, Test = 3669 images, 37 classes. After an 80/20 split of train+val: 2944 train, 736 validation.

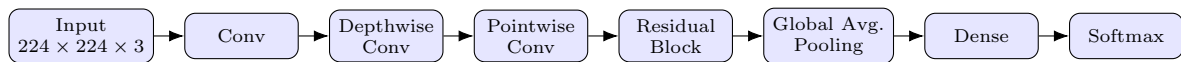
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

Observed: The ImageNet-pretrained MobileNetV2 base has **2,257,984** parameters. All sections below (except fine-tuning) keep this base frozen and train only a small classifier head.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Results

Initialization	Final Val. Accuracy (%)
Zero	1.77
Random	89.40
Xavier	90.49
He	90.22

Best initialization: Xavier.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Results (final val. accuracy)

Configuration	Final Val. Accuracy (%)
No Regularization	89.67
L2	88.72
Dropout	90.35
BatchNorm	90.62

Best regularization: BatchNorm.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Verified in code: $\mu_B = 5.0$, $\sigma_B^2 = 5.0$, $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time (s)
SGD	0.2650	88.72	6	85.5
Momentum	0.0340	90.76	5	73.8
RMSProp	0.0093	90.62	6	77.6
Adam	0.0197	91.17	6	77.0

Best optimizer: Adam.

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Sweep results

Setting	Val. Accuracy (%)
LR = 0.001	89.95
LR = 0.0001	84.10
Batch size 16	90.63
Batch size 32	89.67
Batch size 64	89.81
Dropout 0.0	89.95
Dropout 0.25	89.67
Dropout 0.5	90.35

Best from sweep: LR = 0.001, batch size = 16, dropout = 0.5.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Observed final val. accuracy: Feature Extraction $\approx 90.5\%$; after Fine-Tuning $\approx 90.1\%$ (similar; slight variation run-to-run).

Discussion: Fine-tuning uses a smaller learning rate so that the already-good ImageNet features are adjusted gently instead of being overwritten by large gradient steps.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1 Baseline	91.98	91.30	90.62	91.58	91.58	91.41 $\pm$ 0.50
C2 BestReg	90.76	89.95	89.67	91.71	91.85	90.79 $\pm$ 0.99
C3 HighDropout	92.12	91.17	90.35	91.71	90.62	91.20 $\pm$ 0.73
C4 L2+BN	91.85	90.76	89.40	91.58	91.71	91.06 $\pm$ 1.02

Selected configuration: C1 Baseline (dropout = 0, no L2, no BN, Adam, LR = 10^{-3}) — highest mean and lowest SD.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	91.41%
CV Standard Deviation	0.50%
Test Accuracy	89.89%
Precision	0.9015
Recall	0.8983
F1-score	0.8979
Training Time	112.9 s
Number of Parameters	2,426,725

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	91.41	0.45	89.56	5.1 s
Best Initialization	91.41	0.45	89.56	4.5 s
Best Regularization	90.95	0.88	89.51	5.4 s
Best Optimizer	90.95	0.88	89.51	4.8 s
Best Hyperparameters	91.14	1.34	89.53	5.8 s
Fine-Tuned Model	91.41	0.50	89.89	112.9 s

Justification: C1 Baseline (Xavier init, Adam, frozen MobileNetV2 head) wins on mean CV accuracy and lowest variability. Fine-tuning the selected config on full train data gives the best test accuracy (89.89%) at higher compute cost. Simpler heads with no extra regularization already generalize well because the ImageNet features are strong for this dataset.

14. Source Code

GitHub Repository

15. Plots and Inferences

Plot 1: Training Loss vs. Epoch (Weight Initialization)

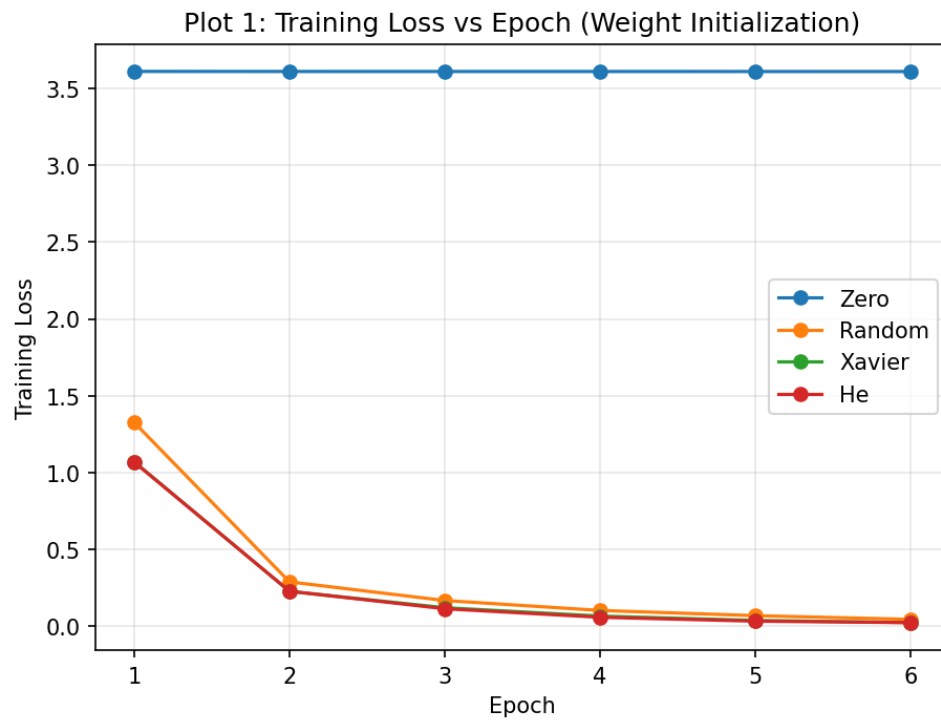


Figure 1: Plot 1: Training Loss vs. Epoch (Weight Initialization)

Inference: Zero initialization did not learn, since the loss stays almost flat. The other methods show a quick drop in loss.

Plot 2: Validation Accuracy vs. Epoch (Weight Initialization)

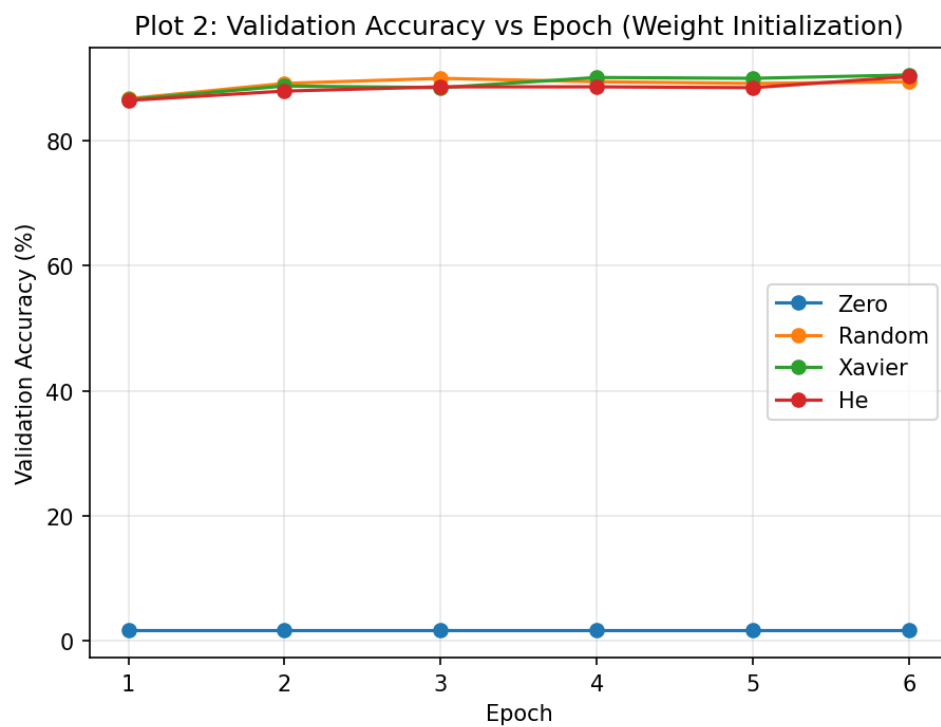


Figure 2: Plot 2: Validation Accuracy vs. Epoch (Weight Initialization)

Inference: Zero initialization stays at only about 1.77% accuracy. The other methods cross 85% very quickly, and Xavier finishes at 90.49%.

Plot 3: Training & Validation Accuracy vs. Epoch (Regularization)

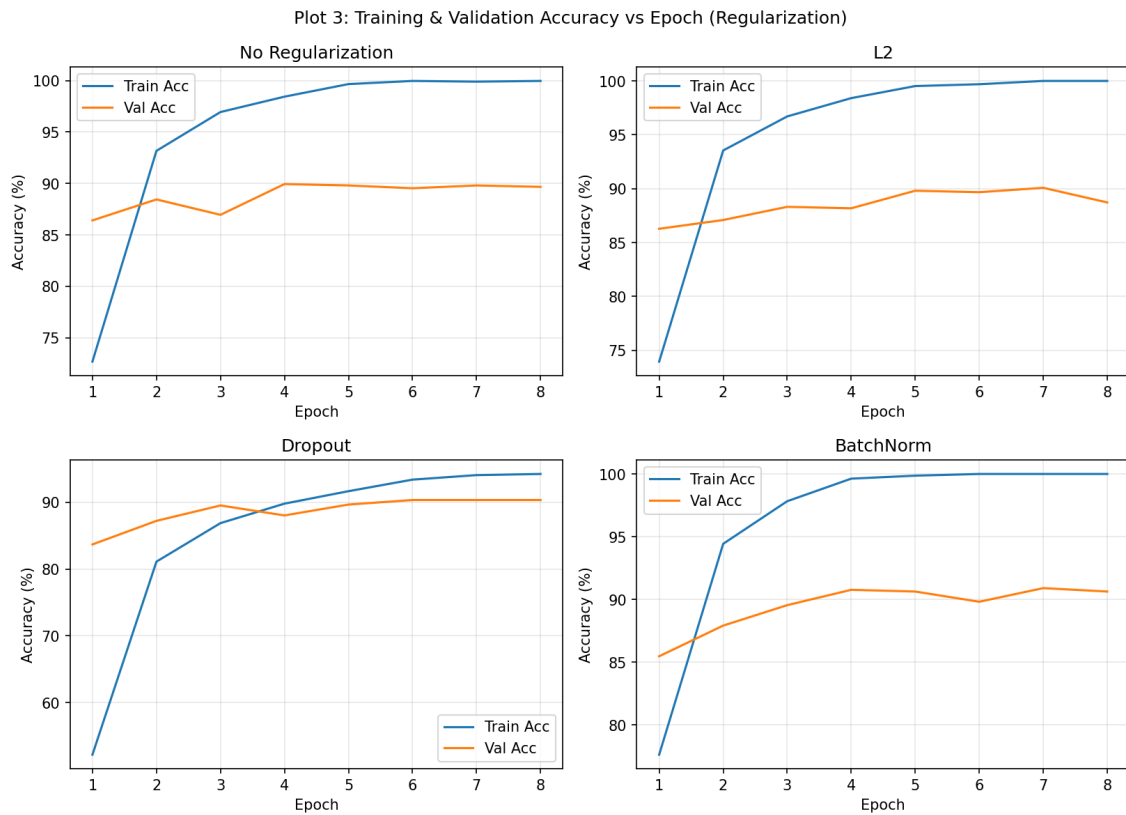


Figure 3: Plot 3: Training & Validation Accuracy vs. Epoch (Regularization)

Inference: All the models get high training accuracy, but the validation accuracy is better with Dropout and BatchNorm. BatchNorm gives the best result here, reaching 90.62%.

Plot 4: Training & Validation Loss vs. Epoch (Regularization)

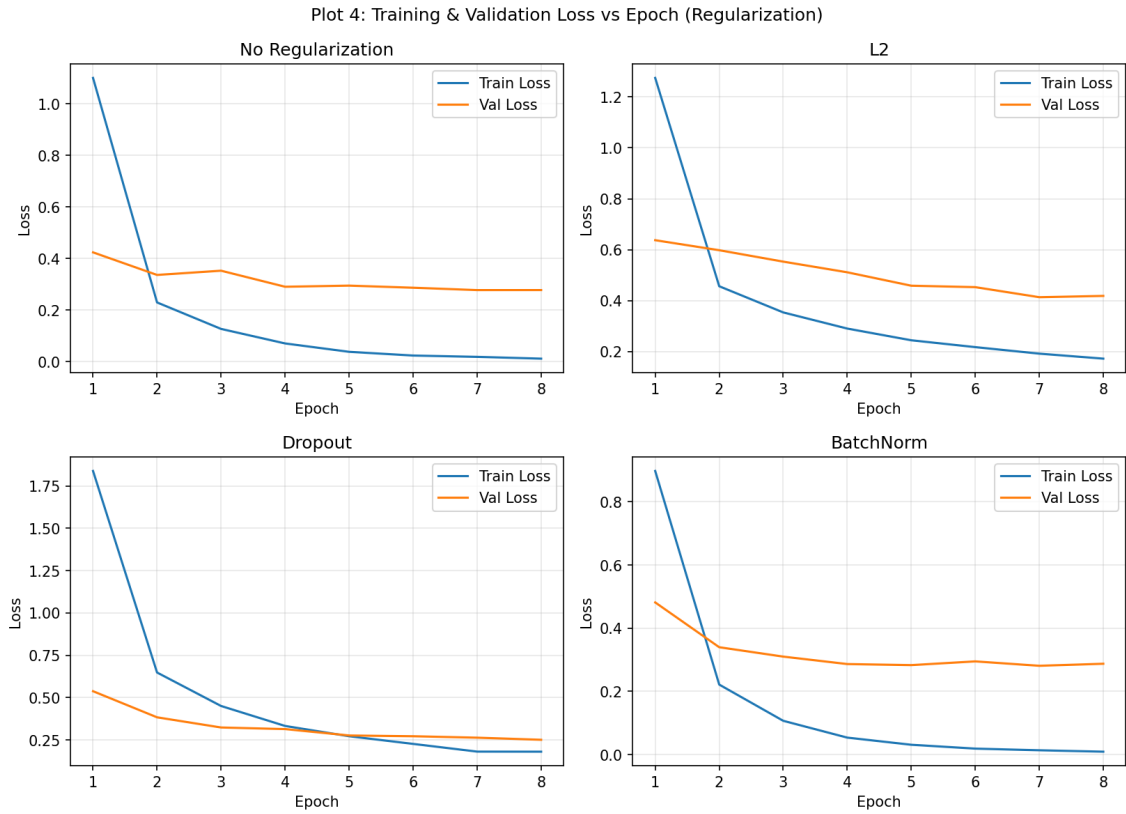


Figure 4: Plot 4: Training & Validation Loss vs. Epoch (Regularization)

Inference: For the setups without regularization, validation loss starts increasing while training loss keeps falling, showing some overfitting. Dropout and BatchNorm keep the validation loss more stable.

Plot 5: With vs. Without Batch Normalization

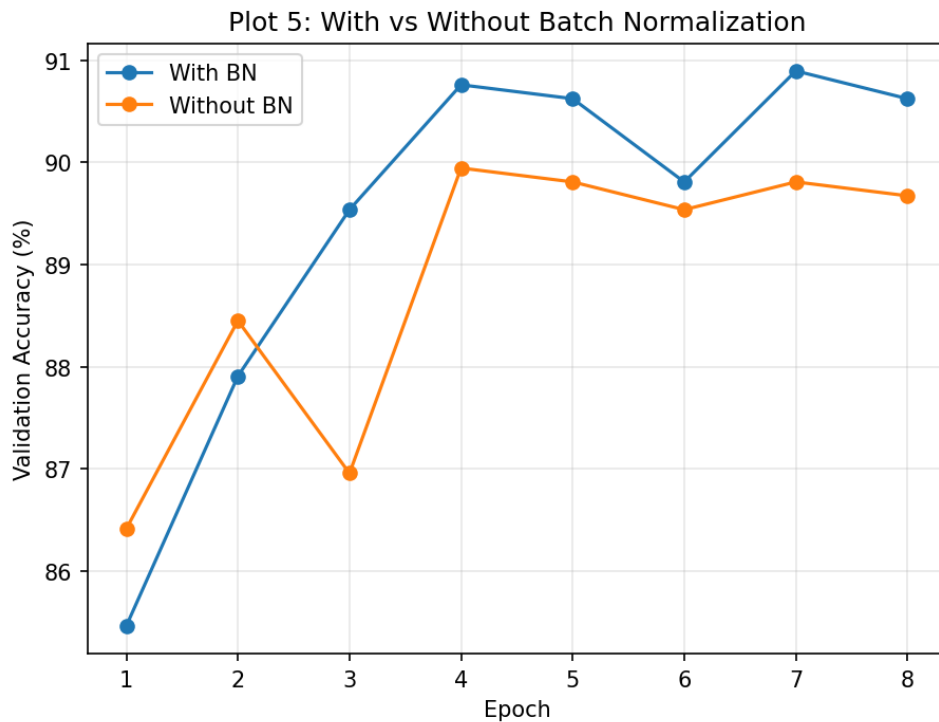


Figure 5: Plot 5: With vs. Without Batch Normalization

Inference: The model with BatchNorm generally has slightly better accuracy and is more stable across the epochs. The model with BatchNorm gives a small improvement.

Plot 6: Training Loss vs. Epoch (Optimizers)

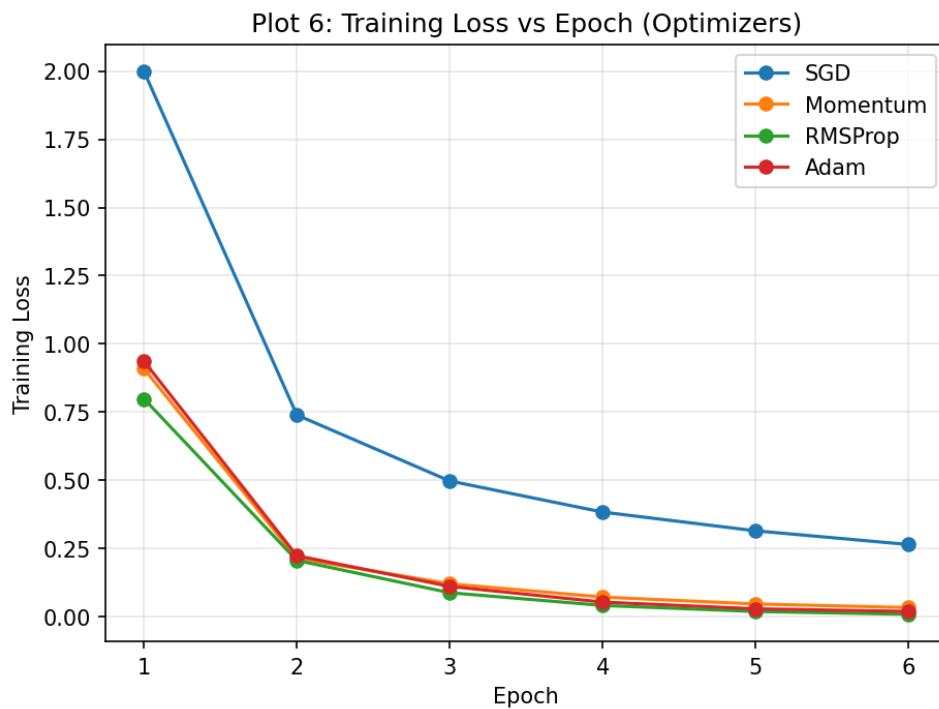


Figure 6: Plot 6: Training Loss vs. Epoch (Optimizers)

Inference: SGD has a much slower decrease in loss compared to the other optimizers.

Plot 7: Validation Accuracy vs. Epoch (Optimizers)

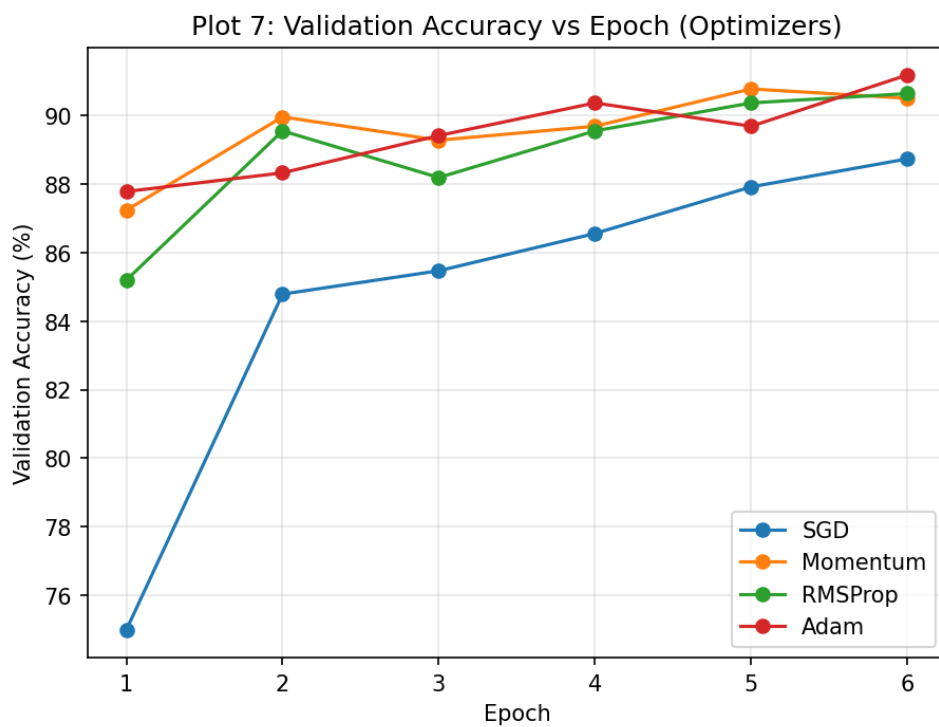


Figure 7: Plot 7: Validation Accuracy vs. Epoch (Optimizers)

Inference: Adam gives the highest validation accuracy at 91.17%. Momentum and RMSProp are fairly close, while SGD gives the lowest accuracy.

Plot 8: Learning Rate vs. Validation Accuracy

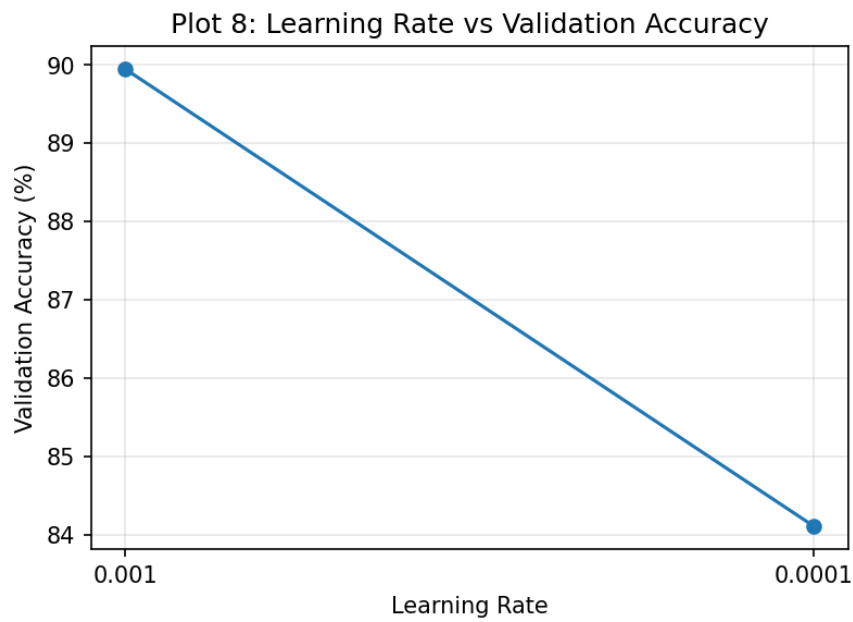


Figure 8: Plot 8: Learning Rate vs. Validation Accuracy

Inference: The learning rate of 0.001 performs better, giving 89.95% accuracy compared to 84.10% for 0.0001. The smaller rate was probably too slow for the given number of epochs.

Plot 9: Batch Size vs. Validation Accuracy

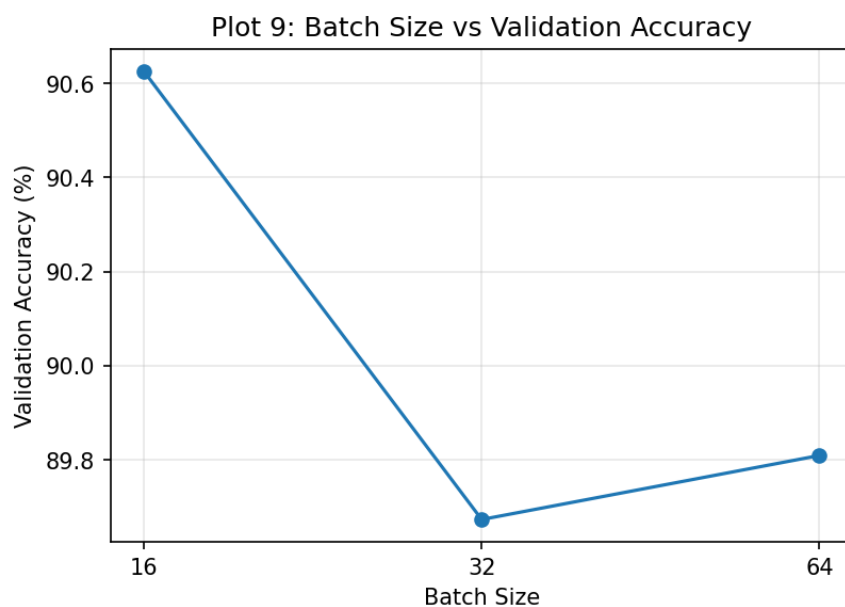


Figure 9: Plot 9: Batch Size vs. Validation Accuracy

Inference: Batch size 16 gives the highest validation accuracy at 90.63%. The smaller batch seems to work better for this model and dataset.

Plot 10: Dropout Rate vs. Validation Accuracy

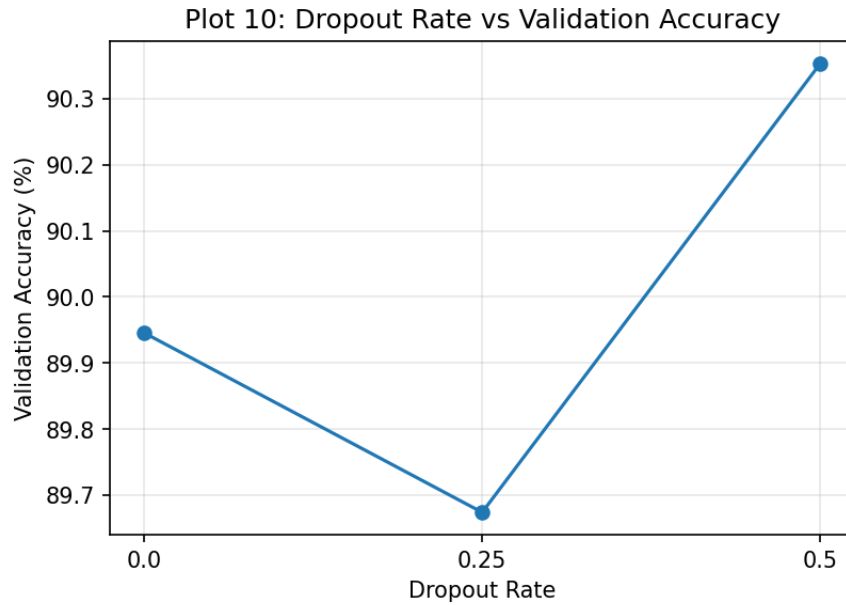


Figure 10: Plot 10: Dropout Rate vs. Validation Accuracy

Inference: A dropout rate of 0.5 gives the best validation accuracy among the tested values. It seems to reduce overfitting without affecting the model too much.

Plot 11: Feature Extraction vs. Fine-Tuning

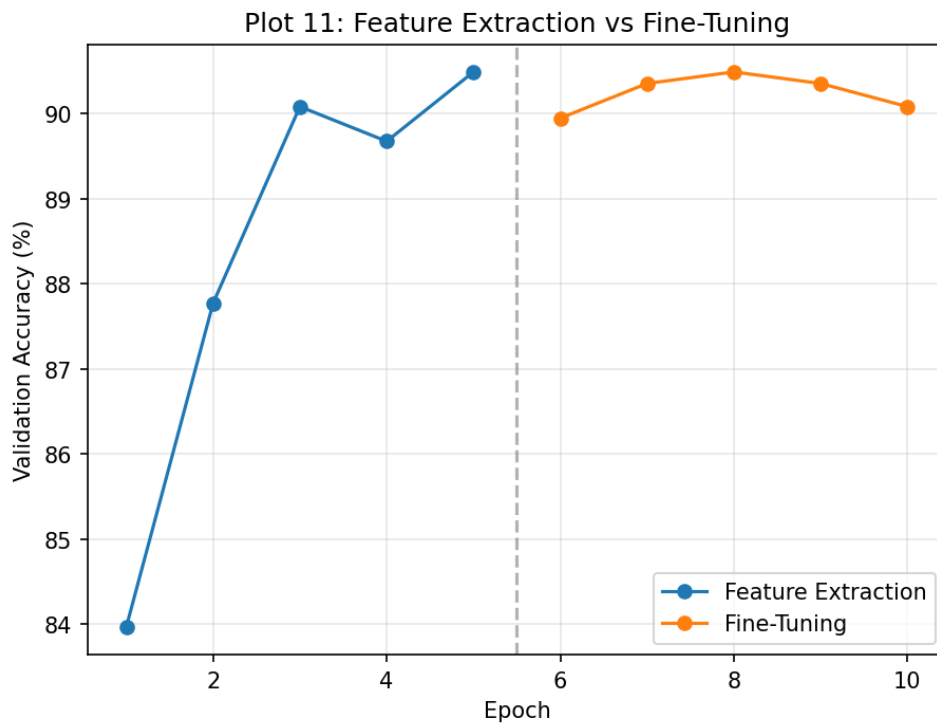


Figure 11: Plot 11: Feature Extraction vs. Fine-Tuning

Inference: Feature extraction gives around 90.5% accuracy, while fine-tuning gives about 90.1%. So, in this case, fine-tuning did not give an improvement.

Plot 12: Training & Validation Loss (Before/After Fine-Tuning)

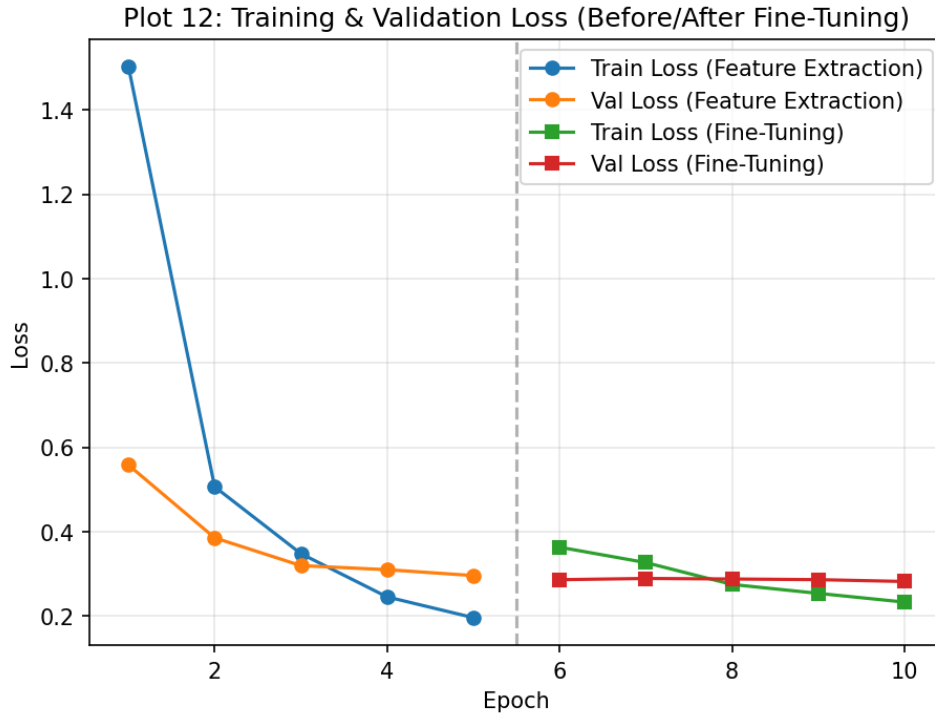


Figure 12: Plot 12: Training & Validation Loss (Before/After Fine-Tuning)

Inference: Using a smaller learning rate during fine-tuning keeps the loss fairly stable. This also avoids changing the pretrained weights too much at once.

Plot 13: 5-Fold Cross-Validation Accuracy

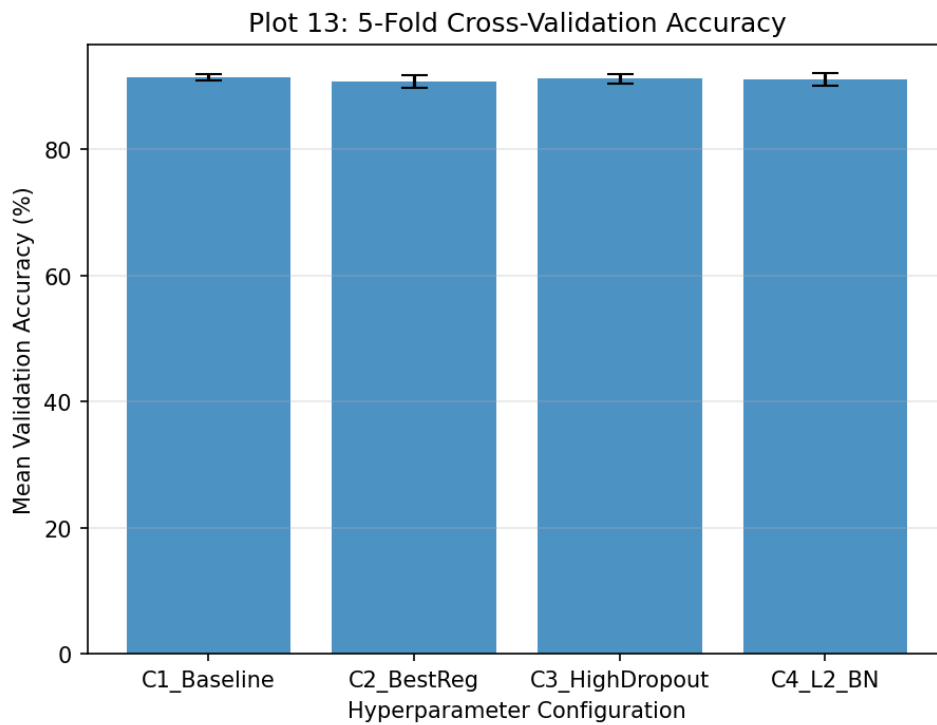


Figure 13: Plot 13: 5-Fold Cross-Validation Accuracy

Inference: C1 Baseline gives the best overall result, with a mean accuracy of 91.41%.

Plot 14: Confusion Matrix (Test Set)

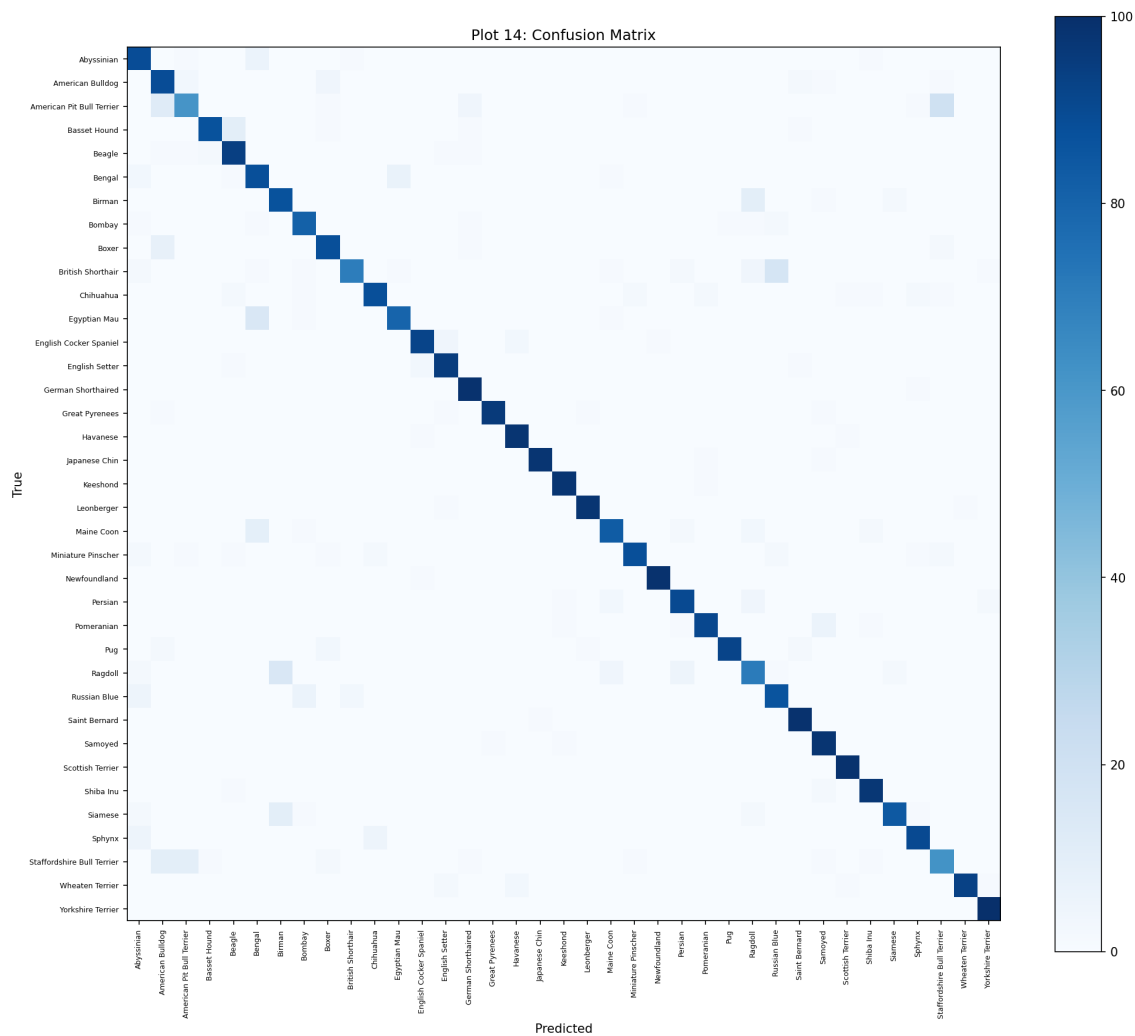


Figure 14: Plot 14: Confusion Matrix (Test Set)

Inference: Some breeds like Saint Bernard and Yorkshire Terrier are classified with very few mistakes. Most confusion is between breeds that look similar, especially American Pit Bull Terrier and Staffordshire Bull Terrier.

Plot 15: Sample Misclassified Images

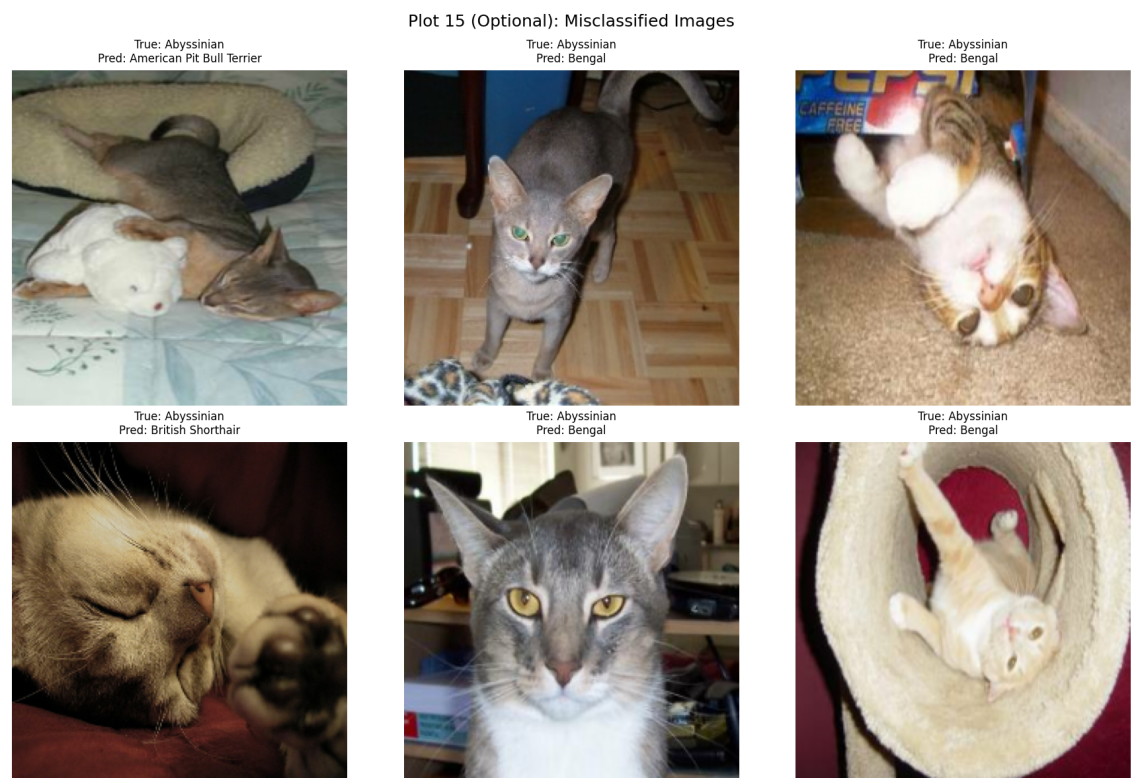


Figure 15: Plot 15: Sample Misclassified Images

Inference: The wrong predictions are mostly from images with poor lighting, cropped views or breeds that look very similar. These make it harder for the model to tell the classes apart.

Plot 16: New Configs vs. Selected Configuration

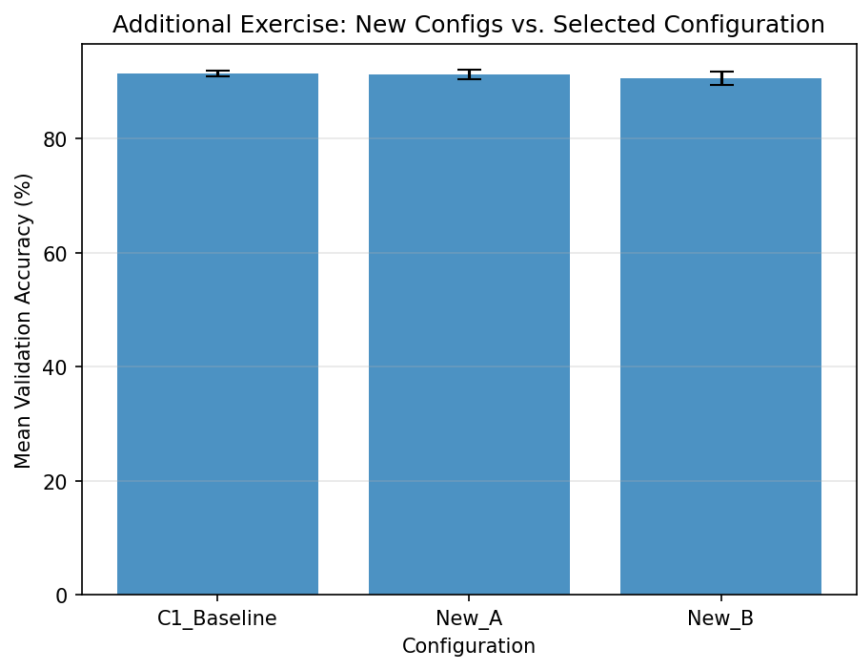


Figure 16: Plot 16: New Configs vs. Selected Configuration

Inference: Neither New_A nor New_B performs better than the selected configuration. New_A is close but has more variation, while New_B performs worse overall.

16. Discussion

- 1. What is the difference between model parameters and hyperparameters?**
Parameters are things like weights and biases that the model learns during training. Hyperparameters like learning rate and batch size are set by user.
- 2. Why is weight initialization important?**
It decides where the model starts learning from. Good initialization helps the network learn properly without the gradients becoming too large or too small.
- 3. Why can zero initialization be problematic for neural networks?**
If all the weights are zero, the neurons behave the same way and get the same updates. So they end up learning the same thing instead of different features.
- 4. Compare Xavier and He initialization.**
Xavier works well for activations like sigmoid and tanh, while He is mainly designed for ReLU. In the experiment, Xavier gave a slightly better result.
- 5. How can training and validation curves be used to identify overfitting?**
If training performance keeps improving but validation performance gets worse, the model is probably overfitting. A rising validation loss along with decreasing training loss indicates this.
- 6. How does Dropout reduce overfitting?**
Dropout randomly turns off some neurons during training. This stops the model from depending too much on a particular set of neurons.
- 7. What is the purpose of Batch Normalization?**
BatchNorm normalizes the activations during training, which makes the learning process more stable. It can also help the model train faster.
- 8. Explain the numerical Batch Normalization example.**
For $[2, 4, 6, 8]$, the mean is 5 and the variance is 5. After normalization, the values become approximately $[-1.342, -0.447, 0.447, 1.342]$.
- 9. What are the roles of γ and β ?**
 γ controls the scale of the normalized values and β shifts them. Both are learned by the network.
- 10. Compare SGD, Momentum, RMSProp and Adam.**
SGD uses the current gradient, while Momentum uses previous updates as well. RMSProp adjusts the updates based on recent gradients, and Adam combines both Momentum and RMSProp.
- 11. What happens when the learning rate is too large?**
The model takes very large steps and can overshoot the minimum. This can make the loss oscillate or even increase.
- 12. What happens when the learning rate is too small?**
The model learns very slowly and may not reach a good solution within the given number of epochs.
- 13. What is the effect of increasing batch size?**
A larger batch gives smoother gradient updates but needs more memory. Smaller batches are noisier but can sometimes generalize better.
- 14. Explain stride and padding.**
Stride is how far the filter moves each time, while padding adds extra pixels around the image. Both affect the spatial size of the output.
- 15. Why is MobileNetV2 computationally efficient?**
MobileNetV2 uses depthwise separable convolutions and inverted residual blocks, which reduce the number of computations compared to standard CNNs.
- 16. What is depthwise separable convolution?**
It first applies a convolution separately to each channel and then uses a 1×1 convolution to combine them. This makes the operation much cheaper.
- 17. What is transfer learning?**
Transfer learning uses a model that has already been trained on a large dataset and adapts it to a new task. This saves both training time and computation.
- 18. Differentiate feature extraction and fine-tuning.**
In feature extraction, the pretrained layers are frozen and only the classifier is trained. In fine-tuning, some pretrained layers are also allowed to update.
- 19. Why is a smaller learning rate generally used during fine-tuning?**
The pretrained model already has useful features, so only small changes required. A large learning rate could change those weights too much.
- 20. Why is K-Fold Cross-Validation useful for hyperparameter selection?**
It tests the model on different train-validation splits instead of depending on just one split. This gives a better idea of the model's average performance.
- 21. Why must the test set remain untouched during tuning?**
Using the test set while tuning can leak information into the model selection process. Keeping it unused gives a

fair final evaluation.

22. **Why should mean and standard deviation both be reported?**

Mean shows the average performance, while standard deviation shows how much the performance changes between folds. So, both accuracy and consistency can be compared.

23. **Is the highest validation accuracy always sufficient to select a model?**

No. Accuracy is important, but factors like model complexity, speed and consistency should also be considered.

17. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

New configurations tested

- **New_A:** dropout = 0.3, no L2, no BN, LR = 5×10^{-4} , Adam
- **New_B:** dropout = 0.4, L2 = 10^{-4} , BN on, LR = 10^{-3} , RMSProp

Configuration	Mean CV Acc. (%)	SD
C1 Baseline	91.41	0.50
New_A	91.25	0.84
New_B	90.65	1.20

18. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

19. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>